Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

ЗВІТ

з лабораторної роботи № 7
з дисципліни «Основи програмування»
«Функції»

Виконав:
студент групи ФЕП-13
Марцінко Н. Р.

Перевірив:
асистент Васюта В. М.

Львів – 2026

Мета роботи:

Вивчити поняття і застосування функцій

2. Завдання 1. Функція перевірки простого числа

За методичкою потрібно створити функцію, яка перевіряє, чи введене з консолі число є простим. Просте число — ціле > 1, яке ділиться лише на 1 і на себе.

2.1. Алгоритм функції is_prime_number

1) Якщо num ≤ 1 — одразу false (0 і 1 не прості).

2) Якщо num == 2 — true (єдине парне просте).

3) Якщо num парне (num % 2 == 0) — false.

4) У циклі for від i = 3 до √num з кроком 2 перевіряємо, чи num ділиться на i; якщо так — false.

5) Якщо жоден дільник не знайдено — true.

Такий підхід суттєво швидший за перевірку всіх чисел до num−1: достатньо йти лише до квадратного кореня і пропускати парні дільники.

2.2. Текст програми (task1.c)

Martsinko > Lab7 > C task1.c

```c
#include <stdio.h>
#include <stdbool.h>

bool is_prime_number(int num) {
    if (num <= 1) return false;
    if (num == 2) return true;
    if (num % 2 == 0) return false;

    for (int i = 3; i * i <= num; i += 2) {
        if (num % i == 0) {
            return false;
        }
    }
    return true;
}

int main(){
    int num;
    printf("Введіть число: ");
    scanf("%d", &num);
    if(is_prime_number(num) == true){
        printf("Число є простим");
    }
    else
    {
        printf("Число не є простим");
    }
    return 0;
}
```

Рис. 1. Код task1.c — функція is_prime_number та main

2.3. Результат виконання

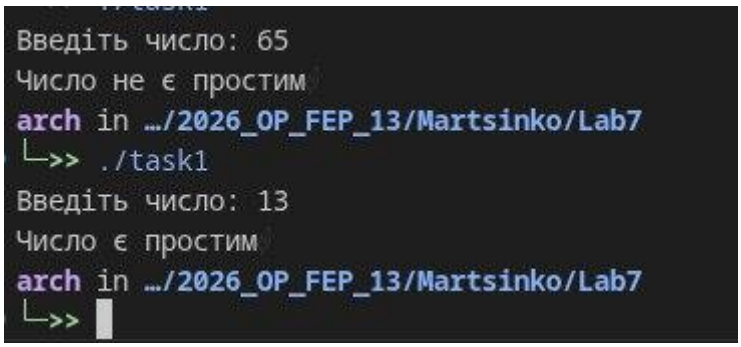


Рис. 2. Вивід: 65 — «Число не є простим»; 13 — «Число є простим»

3. Завдання 2. Рекурсивна сума чисел від номера в журналі до 100

Потрібно рекурсивно обчислити суму цілих чисел від порядкового номера студента в журналі до 100 включно.

3.1. Як працює рекурсія

Для num = 97 ланцюг викликів: recursion(97) → 97 + recursion(98) → 97+98 + recursion(99) → 97+98+99 + recursion(100) → 97+98+99+100. Кожен рівень чекає результату наступного; після досягнення бази стек згортається і суми додаються знизу вгору. Для num = 6 глибина рекурсії — 95 рівнів, що для сучасного стеку цілком прийнятно.

3.2. Текст програми (task2.c)

Martsinko > Lab7 > C task2.c

```c
#include<stdio.h>

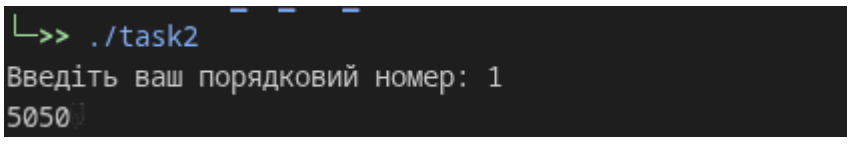
int recursion(int num){
    if (num >= 100){
        return 100;}
    return num + recursion(num + 1);
}

int main(){
    int num;
    printf("Введіть ваш порядковий номер: ");
    scanf("%d", &num);
    printf("%d", recursion(num));

}
```

Рис. 3. Код task2.c — рекурсивна функція recursion

3.3. Результат виконання



Відповідь правильна.

4. Завдання 3. Розв’язок рівняння N·x + A = 0 методом половинного ділення

Рівняння f(x) = N·x + A = 0, де N — номер у журналі, A — вік. Корінь шукається на інтервалі [−100; 100] з точністю ε = 0,00001. Хоча для лінійної функції корінь відомий аналітично (x = −A/N), завдання вимагає саме ітераційного методу бісекції.

4.1. Алгоритм бісекції в програмі

1) Оголошується функція f(x, N, A) = N·x + A.

2) Перевірка умови існування кореня: якщо f(a)·f(b) > 0 на кінцях [−100; 100] — кореня немає (для лінійної з N ≠ 0 це не станеться).

3) Цикл while ((b−a)/2 > eps): mid = (a+b)/2; якщо f(mid) == 0 — точний корінь, break; якщо f(a)·f(mid) < 0 — корінь у [a, mid], інакше в [mid, b].

4) Після виходу з циклу x_approx = (a+b)/2.

4.2. Текст програми (task3.c)

Martsinko > Lab7 > c task3.c

```c
#include <stdio.h>
#include <math.h>

double f(double x, double N, double A) {
    return N * x + A;
}

int main(void) {
    double N, A;

    printf("Введіть порядковий номер у журналі (N): ");
    scanf("%lf", &N);
    printf("Введіть ваш вік (A): ");
    scanf("%lf", &A);

    double a = -100.0;
    double b = 100.0;
    double eps = 0.00001;

    if (f(a, N, A) * f(b, N, A) > 0) {
        printf("На відрізку [%.0f, %.0f] кореня немає.\n", a, b);
        return 1;
    }

    double mid;

    while ((b - a) / 2.0 > eps) {
        mid = (a + b) / 2.0;

        if (f(mid, N, A) == 0.0) {
            break;
        }

        if (f(a, N, A) * f(mid, N, A) < 0) {
            b = mid;
        } else {
            a = mid;
        }

    }

    double x_approx = (a + b) / 2.0;

    printf("\nРезультат методу бісекції:\n");
    printf("Знайдений корінь x: %.5lf\n", x_approx);

    return 0;
}
```

Рис. 5. Код task3.c — функція f і цикл бісекції

4.3. Результат виконання

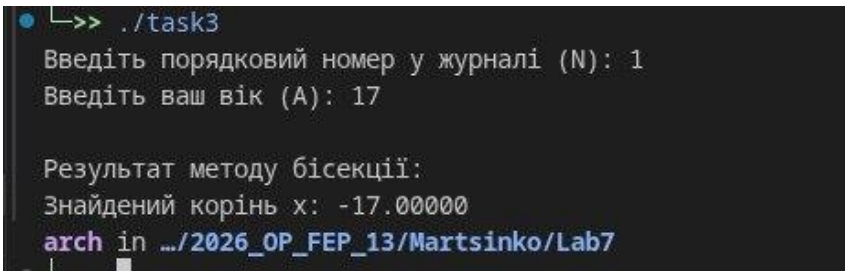


Рис. 6. Вивід: N = 1, A = 17 → знайдений корінь x = −17,00000

Аналітичний корінь: $x = -A/N = -17/1 = -17$. Метод бісекції дав −17,00000 — збіг у межах ε. Кількість ітерацій для інтервалу довжиною 200 і $\varepsilon = 10^{-5}$ оцінюється як $\log_2(200/10^{-5}) \approx 24$ кроки. Тобто комп'ютер справді десятки разів ділить відрізок навпіл, хоча для лінійного рівняння достатньо однієї арифметичної операції.

Висновки

Ця лабораторна робота виявилася помітно складнішою за попередні, оскільки кожне завдання вимагало обережності з логікою та граничними умовами. Перші два завдання виконалися без проблем, а ось третє не дуже. Шукати розв'язок елементарного лінійного рівняння методом половинного ділення виглядає як зайве ускладнення собі життя, адже комп'ютер замість однієї миттєвої арифметичної дії змушений десятки разів навпомацки ділити відрізок навпіл і звужувати межі.